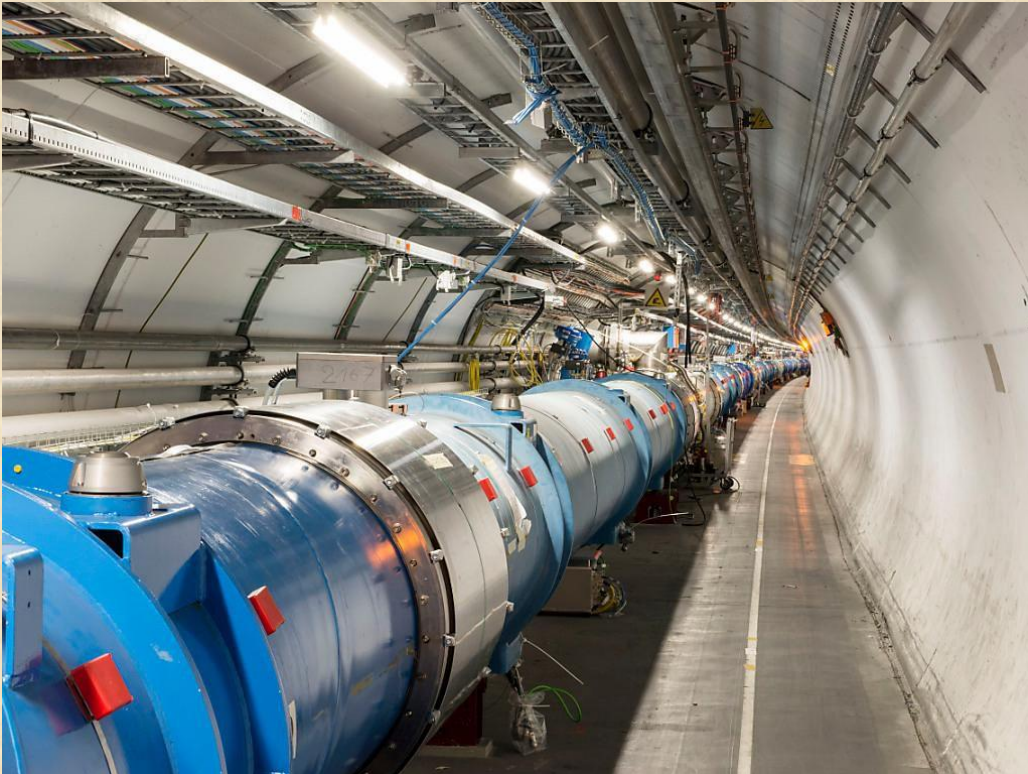


Profiles for simplicity and guarantees



Bjarne Stroustrup

Columbia University

www.Stroustrup.com

Safety and quality

- We need “safety”, elegance, and efficiency
 - Good programmers can write modern C++ to meet those demands
 - Proof: the many quality systems we all rely on every day
- But
 - It is easy to get lost in complexities
 - 18.4 million developers can’t all consistently deliver top quality
 - Our developer community is growing fast
 - Implies education needs
- We need guarantees
 - Well-defined definitions of “safety”
 - Enforced by well-implemented tools (e.g., compilers and libraries)
- We need to preserve C++’s flexibility, efficiency, and style
 - Enable gradual adoption
 - Encourage the use of “modern C++”

- “Safety is easy. It’s granting access that’s hard”
 - David Wheeler (to Bjarne Stroustrup about 1978)

Most of this talk focus on making effective techniques safer and simpler than currently, without loss of expressiveness and efficiency.



Two major problems

- C++ is complex
 - It is for every individual user
 - Complexity is needed for serving a huge range of uses
 - Complexity is partly the effect of long-term success
- All of C++ cannot be guaranteed “safe”
 - For any reasonable definition of “safe”
 - Like for every language capable of effective systems programming
 - Worries about this are often exaggerated, but yes
 - Problems are often made worse by poor education and inappropriate styles of use, but yes
 - C++ is successfully used in many critical systems
- We can address those problems
 - Without breaking good C++
 - Without violating the zero-overhead principle

A profile

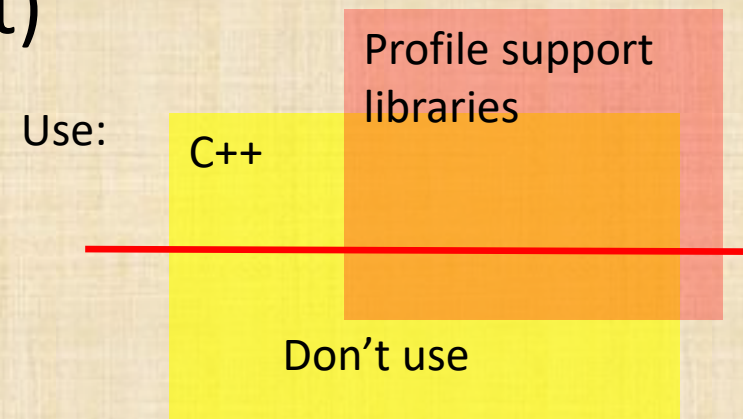
- Is
 - A set of coherent guarantees of desirable properties
 - Enforced by a combination of static analysis and run-time checks
- Dramatically
 - Simplifies the use of C++
 - Increases the safety of C++
 - For some definition of “safety”
 - Simplifies the teaching of C++
- Examples
 - Every object is initialized before use
 - No access is through a dangling pointer

Profile aims

- Offer guarantees
 - Enforced by tools
 - Was always the ideal for C++
 - Not just rely on developers following rules and adding annotations
 - correctly, completely, and consistently
 - As required by organizations and regulatory bodies
- Unverified (“Trusted”) code should be minimized and made explicit
 - Some is unavoidable
 - “All or nothing” is sloppy and error-prone
- Many different guarantees are needed
 - One definition cannot serve everyone
 - Enable gradual adoption
- The resulting code is always ISO C++
 - Identical meaning except possibly for what was UB before enforcing the profile

Profile strategy (subset of superset)

- Do nothing until the user asks for it in code
 - E.g., initialization guarantees
 - E.g., guarantee the absence of dangling pointers
 - E.g., guaranteed run-time range checks
- Add supporting libraries
 - To make it easy to follow the rules needed to deliver the requested guarantee
- Impose simplification rules to make local static analysis feasible
 - For the required profile
 - Static analysis of arbitrary code is impossible
 - Global static analysis is unaffordable
- Provide attributes to speed up compilation
 - Only attributes that can be verified
- Use run-time checks where static analysis is impossible or too expensive
 - Provide violation handling alternatives to match diverse user needs



What Profiles are not

- Not changing the meaning of a program
 - Beyond possibly giving meaning to (former) UB when enforced
- Not changing the base language (ISO C++)
 - Beyond adding attributes and libraries
- Not introducing dialects
 - Just enforcing a subset of superset as good developers usually do
- Not whole-program analysis
- Not purely static analysis
 - Though the initialization and invalidation profiles are
- Not purely run-time checks
- Not a borrow-checker
- Not relying on a garbage collector
- Not introducing unverifiable attributes that would allow users to introduce errors
- Not a closed set
 - Not every profile will be in the ISO C++ standard
- Not a brand-new untried idea
 - The synthesis of techniques tried for decades (guidelines, static analysis, run-time assertions, invariants)

hoping to counter
some myths

Where to start – problems to eliminate early

- Eliminate failing to initialize
 - If we read or write memory before initialization, no guarantees hold
- Eliminate access through dangling pointers
 - If we read or write through dangling pointers, no guarantees hold
- Catch out-of-range access and access through nullptr
 - If we access out of range, no guarantees hold
- This talk
 - Is about eliminating initialization and dangling pointers problems
 - Implementations exist
 - Outlines the long-term aims of Profiles
 - Complete type-and-resource safety



Profiles – the framework

- Directives
 - `[[profiles::enforce(P)]]` for this module or scope
 - `[[profiles::require(P)]]` for an imported module or scope
 - `[[profiles::suppress(P)]]` for a module or a scope
 - `[[profiles::exempt(P,header)]]` exempt a header from the checking of a Profile
- We need some standard profiles
 - Initialization
 - Type
 - Arithmetic
 - Range
 -
- Users can add their own profiles.
 - Non-standard
 - Domain specific
 - Stronger/weaker

Implementations in

- GCC and Clang

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3589r3.pdf>
Gabriel Dos Reis. C++ Profiles: The Framework. 2026-07-21

The initialization profile

- Guarantee
 - Every object is initialized before use
 - Every object is destroyed before its memory is reused
 - Every object is initialized and destroyed once only



- CppCon 2026

Remember:
No rule holds when there is a data race.
Concurrency is another profile

caveat13

Manipulating uninitialized memory

- Much code depends critically on manipulating uninitialized memory

- Memory pools
- Stack allocators
- Communication buffers
- **std::vector**
- Delayed initialization

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p4222r2.pdf>

B. Stroustrup: An initialization profile. 2026-08-12.

R3 and implementations will match.

- It wouldn't be C++ if we couldn't do that well
- Other languages rely on
 - “outsourcing” the problem to C or C++
 - Suppressing all checking
- A constructor turns uninitialized memory into an object
- A destructor turns an object into uninitialized memory

The initialization profile

- We build and maintain very complex code related to initialization
 - Often for very good reasons
- The initialization profile simplifies such code
 - Makes intent explicit and obvious
 - Enables and simplifies analysis

Be precise – say what you mean

- Consider: **X** **x**;
- Is **x** initialized?
 - **X** might have a default constructor, or not
 - **X** might be in global scope and zero-initialized, or be local and uninitialized
 - **X** might be a class member, but that we will look at later
 - If **x** is uninitialized was that deliberate? The compiler doesn't know.
- Be specific
 - **X** **x** ; *// x must be implicitly initialized*
 - **X** **y** **[[uninit]]**; *// y must be uninitialized*
- With the initialization profile enforced
 - An error if **x** is not initialized by default
 - An error if **y** is initialized by default (double initializations can be very bad)
 - The profile checks that **y** is initialized before use

GCC


Add... ▾ More ▾ Templates
Share ▾ Policies  Other ▾

C++ source #1 

A ▾



+ ▾

v





C++ ▾

```

1  [[profiles::enforce(std::init)]];
2
3  struct X { int y; };
4
5  X x0;    // OK: static init
6
7  void f()
8  {
9      X x1;    // error
10     X x2 = { 2 };
11     X x3 [[uninit]];
12     int y = x3.y;    // error
13 }

```

Output of x86-64 gcc (P4324 contracts) (Compiler #1)  

A ▾

☒ Wrap lines

 Select all

```

<source>: In function 'void f()':
<source>:9:11: error: local variable 'x1' not initialized and not
marked '[[uninit]]' under the 'std::init' profile
    9 |         X x1;           // error
      |         ^~
<source>:12:13: error: member 'X::y' of 'x3' read before it is
definitely assigned, under the 'std::init' profile
   12 |         int y = x3.y;    // error
      |             ^
Compiler returned: 1

```

x86-64 gcc (P4324 contracts) (Editor #1)  

x86-64 gcc (P4324 contra ▾   Compiler options... ▾

A ▾









+ ▾



```

1  <Compilation failed>
2

```

Clang


Add... ▾ More ▾ Templates ⓘ
Share ▾ Policies ⓘ Other ▾

C++ source #1 

A ▾







C++ ▾

```

1  [[profiles::enforce(std::init)]];
2
3  struct X { int y; };
4
5  X x0;    // OK: static init
6
7  void f()
8  {
9      X x1;    // error
10     X x2 = { 2 };
11     X x3 [[uninit]];
12     int y = x3.y;    // error
13 }

```

Output of x86-64 clang (std::init profile - P4222) (Compiler #1)  

A ▾

☒ Wrap lines

Select all

```

<source>:9:4: error: variable 'x1' must be initialized or marked
'[[uninit]]' under profile 'std::init'
[-Wprofile-std-init-uninit-decl]
    9 |         X x1;           // error
      |         ^
<source>:12:13: error: read of a subobject of an '[[uninit]]' object
accesses uninitialized memory under profile 'std::init'
[-Wprofile-std-init-uninit-read]
    12 |         int y = x3.y;    // error
      |                   ^
2 errors generated.
Compiler returned: 1

```

x86-64 clang (std::init profile - P4222) (Editor #1)  

x86-64 clang (std::init pro ▾





-std=c++23 -pedantic-errors ▾

A ▾








1

<Compilation failed>

Stroustrup - Profiles - CppCon 2026

18

What about delayed initialization?

- Simplest example

```
X x [[uninit]];
```

```
X y = z;
```

```
construct_at(&x,z);
```

```
// OK, and after this x is no longer uninitialized
```

```
construct_at(&y,z2);
```

```
// Error: double initialization
```

Same for destruction

- Types without constructors can be initialized by assignment
- Delayed assignment
 - E.g., for an array using a loop
 - An error unless we can prove that every object is initialized
 - Common current practice in many languages

What about delayed initialization?

- Simplest example

```
X x [[uninit]];
```

```
X y [[uninit]];
```

- Every path leading to a use must be proven to initialize
 - In general, determining whether something is initialized is impossible or expensive
 - Use local static analysis only

```
if (b)
```

```
    construct_at(&x,z1);
```

```
else
```

```
    construct_at(&x,z2);
```

```
// OK
```

```
if (b2)
```

```
    construct_at(&y,z1);
```

```
// error: no else
```

Same for destruction



Add... More Templates

Share Policies Other

C++ source #1

A B + - V

C++

```

4
5 int main(int argc, char** argv)
6 {
7     int x [[uninit]];
8     int y [[uninit]];
9
10    if (argc > 1)
11        std::construct_at(&x, 1);
12    else
13        x = 2;
14
15    if (argc > 1)
16        std::construct_at(&y, 42);
17    return x+y;
18 }
19

```

x86-64

Output of x86-64 gcc (P4324 contracts) (Compiler #1)

x86-64 gcc (P4324 contra

lines Select all

<source>: In function 'int main(int, char**)':
<source>:17: error: 'y' read before it is definite

```

17 |     return x+y;
    |             ^^

```

Compiler returned: 1

Passing uninitialized memory around

- We often pass uninitialized memory around to be initialized
- Be explicit
 - Being initialized is as ever the default assumption

```
void f(X* p [[ref_to_uninit]]);    // p must point to something uninitialized
void g(X* p);                      // q must point to something initialized
```

```
void tst()
{
    X x [[uninit]];
    f(&x);
    g(&x);    // error

    X y = 7;
    f(&y);    // error
    g(&y);
}
```



Add... More Templates

Share Policies Other

C++ source #1

A B + v C++

```
1  [[profiles::enforce(std::init)]];
2
3  struct X { int y; };
4
5  void f(X* p [[ref_to_uninit]] );
6  void g(X* p);
7
8  void tst()
9  {
10     X x [[uninit]];
11     f(&x);
12     g(&x);      // error
13
14     X y = { 7 };
15     f(&y);      // error
16     g(&y);
17 }
```

Output of x86-64 gcc (P4324 contracts) (Compiler #1)

A Wrap lines Select all

```
<source>: In function 'void tst()':
<source>:15:6: error: argument 1 to 'void f(X*)' must refer to '[[uninit]]' memory,
matching its '[[ref_to_uninit]]' parameter, under the 'std::init' profile
15 |     f(&y);      // error
    |     ^^^~
<source>:12:6: error: address of '[[uninit]]' variable 'x' is taken here before it is
provably initialized, under the 'std::init' profile
12 |     g(&x);      // error
    |     ^^^~
<source>:10:7: note: 'x' is declared '[[uninit]]' here
10 |     X x [[uninit]];
    |       ^
Compiler returned: 1
```

x86-64 gcc (P4324 contracts) (Editor #1)

x86-64 gcc (P4324 contra Compiler options...

A Output... Filter... Libraries Overrides + Add new... Add tool...

```
1  <Compilation failed>
2
3  # For more information see the output window
```

Passing uninitialized memory around

- A `[[ref_to_uninit]]` may be used in a way that doesn't initialize every element
 - E.g. memory held by a **vector**

```
X x [[uninit]];
void f(X* p [[ref_to_uninit]]);    // p must point to something uninitialized
// ... here *p is not considered initialized...
f(&x);
X z = x;        // error: x is still [[uninit]]
```

- We can require that all elements must be initialized
 - A `[[must_init]]` is a `[[ref_to_uninit]]` that must initialize all elements

```
X x [[uninit]];
void f(X* p [[must_init]]);        // after the call, p points to something uninitialized
// ... here *p is initialized...
f(&x);
X z = x;        // OK: x is initialized
```

Vector

- **Vector_memory** manages uninitialized slots for a Vector
 - a *[[ref_to_uninit]]* cast to a pointer yields a *[[ref_to_uninit]]*

```
template<class T>
struct Vector_memory {
    T* elem [[ref_to_uninit]] = nullptr;           // points to some uninitialized memory
    int no_of_elements = 0;
    int no_of_slots = 0;

    Vector_memory(int ne, int ns)
        : elem{(T*)my_malloc((ne+ns)*sizeof(T))}, // allocate some memory without initializing
          no_of_elements{ne},
          no_of_slots{ns} { }

    ~Vector_memory() { free(elem); }
};
```

```
template<class T>
class Vector {
    Vector_memory<T> mem;
```

```
public:
```

```
    Vector() {}
```

```
    Vector(int ne, int ns, const T& val = T{})
```

```
    : mem{ne,ns}
```

```
    { uninitialized_fill(mem.elem,mem.elem+ne,val); }           // initialize all slots
```

```
    void reserve(int newsz);           // make Vector hold a mix of elements and unininitialized slots
```

```
    T& operator[](int i) {
```

```
        if (0<=i && i<mem.no_of_elem)
```

```
            return *now_init(mem.elem[i]);           // this slot has been initialized
```

```
            throw Bad_index{};
```

```
    }
```

```
    ~Vector() { destroy(now_init(mem.elem), mem.ne); }
```

```
    // ...
```

```
};
```

Mem:

objects

Uninitialized slots

ne

Very specific profile suppression



Add... More Templates

Share Policies Other

C++ source #1

Save/Load Add new... Vim CppInsights Quick-bench

C++

```

21 template<class T>
22 class Vector {
23     Vector_memory<T> mem [[uninit]];
24 public:
25     Vector() {}
26
27     Vector(int ne, int ns, const T& val = T{})
28         : mem{ ne,ns }
29     { uninitialized_fill(mem.elem, mem.elem + ne, val); } // initialize a
30
31     void reserve(int newsz); // make Vector hold a mix of elements and unin
32
33     T& operator[](int i) {
34         if (0 <= i && i < mem.no_of_elem)
35             return *now_init(mem.elem[i]); // this slot has been initia
36         throw Bad_index{};
37     }
38
39     ~Vector() { destroy(now_init(mem.elem), mem.ne); }
40     // ...
41 };

```

Output of x86-64 gcc (P4324 contracts) (Compiler #1)

Wrap lines Select all

Compiler returned: 0

x86-64 gcc (P4324 contracts) (Editor #1)

x86-64 gcc (P4324 contra Compiler options..

Settings Filter Editor Actions

1 <No assembly to display (~18 lines filtered)>

2

Passing uninitialized memory around

- The C and C++ types systems do not distinguish initialized and uninitialized
 - The initialization profile must
- Consider allocators
 - **malloc()** returns a **void*** pointing to uninitialized memory
 - **calloc()** returns a **void*** pointing to zero-initialized memory
- The obvious solution would be to define
 - **void* malloc [[ref_to_uninit]] (std::size_t size);**
 - **void* calloc(std::size_t num, std::size_t size);**
- Can implementers of profiles add attributes to functions?
 - Their own functions? yes
 - C++ standard library functions? Yes, for standard attributes
 - Someone else's functions? Maybe
- Casting a **[[ref_to_uninit]]** yields a **[[ref_to_uninit]]**
 - **T* p = (T*)malloc(10*sizeof(T));** *// p is [[ref_to_uninit]]*

Where we can't add attributes,
we must add differently named
functions

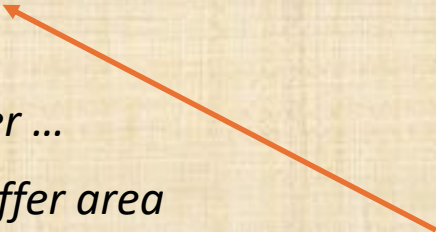
Sketch of a common-style of input operation

```
std::byte* inbuf [[ref_to_uninit]] = (std::byte*) my_malloc(imax); // memory from some buffer pool

class Message { /* operations to read typed members from a byte buffer rep */ };

int read(std::byte* p [[ref_to_uninit]], int max); // read at most max bytes into *p; return the number of bytes read

while(live) { // read into inbuf and process the messages stored as bytes
    int n = read(inbuf,imax); // some input function
    span<Message> messages { (Message*)now_init(inbuf), n/sizeof(Message) }; // this buffer contains Messages
    for (Message& m: messages) {
        // ... process a message from the buffer ...
        destroy(m); // maybe scrub buffer area
    }
}
```



Very specific profile suppression

C++ source #1 

A ▾  Save/Load + Add new... ▾ Vim  Cpplnsights  Quick-bench

 C++ ▾

```

7
8 [[ ref_to_uninit]] void* my_malloc(int);
9 int imax = 100;
10
11 std::byte* inbuf [[ref_to_uninit]] = (std::byte*) my_malloc(imax);
12
13 class Message {
14     // operations to read typed members from a byte buffer rep
15 };
16
17 int read(std::byte* p [[ref_to_uninit]], int max);
18
19 void destroy(Message&);
20
21 void tst(int live)
22 {
23     while(live) { // read into inbuf and process the messages stored as bytes
24         int n = read(inbuf,imax); // some input function
25         std::span<Message> messages { (Message*)now_init(inbuf), n/sizeof(Message) };
26         for (Message& m: messages) {
27             // ... process a message from the buffer ...
28             destroy(m); // maybe scrub buffer area
29         }
30     }
31 }

```

Output of x86-64 gcc 

A ▾ ☒ Wrap lines 

Compiler
returned: 0

x86-64 gcc (P4324) 

x86-64 gcc (P4324 con

  Compiler opt

A ▾  ▾  
 + 

 Output (0/0) | already

Static objects

- Consider

```
int f() { extern int y; return y; } // f.c
int x = f();
int g() { extern int x; return x; } // g.c
int y = g();
```

Either order of initialization leads to an uninitialized variable being read.
- Non-local static objects must be initialized at compile time or link time.
 - No run-time initialization is allowed for such objects.
- There are simple ways to obey this rule:
 - Don't use global variables.
 - Use only **constexpr** global variables.
 - Define constructors to be constexpr.
 - Wrap statics in functions that guarantee initialization before use
- Too strict?
 - To be reviewed when we have more experience
 - Until then, suppress initialization profile if it seems necessary

Do you know all those initialization rules?

- Do you remember always to use all of them?
- Do you find it easy to spot violations when reviewing other people's code?
- Probably not
 - Initialization errors are not uncommon
- The initialization profile
 - Catch errors for you
 - Allows you to be explicit when managing uninitialized memory
 - Allows you to stay ignorant of subtle rules until you accidentally violate them
- Using the initialization profile makes programming simpler as well as safer
 - Essentially all the implementation effort goes into the complex yet essential cases
 - When you consistently initialize every object you can forget about it
 - You can validate your code on one compiler and then run it unchanged on another

Initialization profile summary

- An object must be initialized before its use
 - And destroyed before any reuse of its memory
- Initialization profile attributes
 - `[[uninit]]` an object needs to be initialized before use
 - `[[ref_to_uninit]]` must refer to an `[[uninit]]`
 - `[[must_init]]` a `[[ref_to_uninit]]` that must be initialized
- An attribute appears after the name of the name it refers to
- A `[[ref_to_uninit]]` cast yields a `[[ref_to_uninit]]`
- A function to indicate previously uninitialized memory has been initialized
 - `T* now_init(T* p [[ref_to_uninit]]);` asserts that `*p` has been initialized
 - `T* now_uninit [[ref_to_uninit]] (T* p);` asserts that `*p` now is uninitialized
- A namespace static must be compile-or-link-time initialized
- Zero run-time overhead (all enforced by static analysis)

Profile attributes are verified

- The attributes are there to simplify the work of the compiler
 - To make the programmer's intent explicit
- They are validated for correctness
 - Using local information only
- They are **not** the kind of annotations that can introduce run-time errors

```
int x [[uninit]] = 7;
```

```
// error: x is initialized
```

```
int y = 9;
```

```
int* p [[ref_to_uninit]] = &y;
```

```
// error: y is initialized
```

```
int f(int* arg [[must_init]]) { return 0; }
```

```
// error: f() doesn't initialize
```

Why not make `[[uninit]]` part of the type?

- A profile doesn't change the meaning of a program
 - Except sometimes giving meaning to undefined behavior
- If `[[uninit]]` were part of the type
 - You could overload on it and get different results compiling with and without the initialization profile
 - You might not be able to compile a program on a compiler that ignored `[[uninit]]`

Invalidation profile



- Guarantee
 - No access through a “dangling pointer”
 - That is, through a pointer that no longer points to a valid object
 - A “pointer” here means anything that identifies an object
 - **nullptr** must/will be handled by run-time checks in another profile
- Based on B. Stroustrup: [Profile invalidation – eliminating dangling pointers](#). P3446R0. 2024-10-14.
- B. Stroustrup: [An initialization profile](#). P4222R2. 2026-08-12. R3 and implementations will match.
- Implementations: GCC and Clang
- Partial implementation: the C++ Core Guidelines invalidation profile
- Partial implementation: H. Sutter: Lifetime safety: Preventing common dangling. P1179R1. 2019-11-22.

Dangling pointers

- We ***must*** eliminate access through dangling pointers
 - Or type safety is compromised
 - Or memory safety is compromised
 - Or resource safety is compromised
- Some dangling involves just one scope
 - Relatively easy to catch
 - But still require simple flow analysis
- Some dangling involves many functions
 - Can still be handled by local static analysis
 - Analysis can be significantly speeded up by making intent explicit using attributes
 - Attributes can be verified
 - As usual, not a potential bug source

What are “Pointers”?

- Here, “pointer” means any object that directly identify an object
 - Pointers
 - References
 - Classes with pointer members
 - Lambdas with capture-by-reference
 - **jthreads** holding pointers (they are classes in a defined scope)
 - **unique_ptr**s and **shared_ptr**s (they are classes with a single logical pointer member)
 - Iterators
 - Pointers to pointers
 - References to pointers
 - Arrays of pointers

Local pointers

- Consider

```
void f(int x, int y)
{
    int* p = new int{7};
    int* q = new int{8};

    delete q;
    if (0 < x) *q = 11;    // possible disaster
    *q = 9;               // error: likely disaster

    if (0 < x) delete p;
    *p = 10;              // possible disaster
    if (0 < y) *p = 1;     // possible disaster
    // possible leak of *p
}
```

- Local static analysis

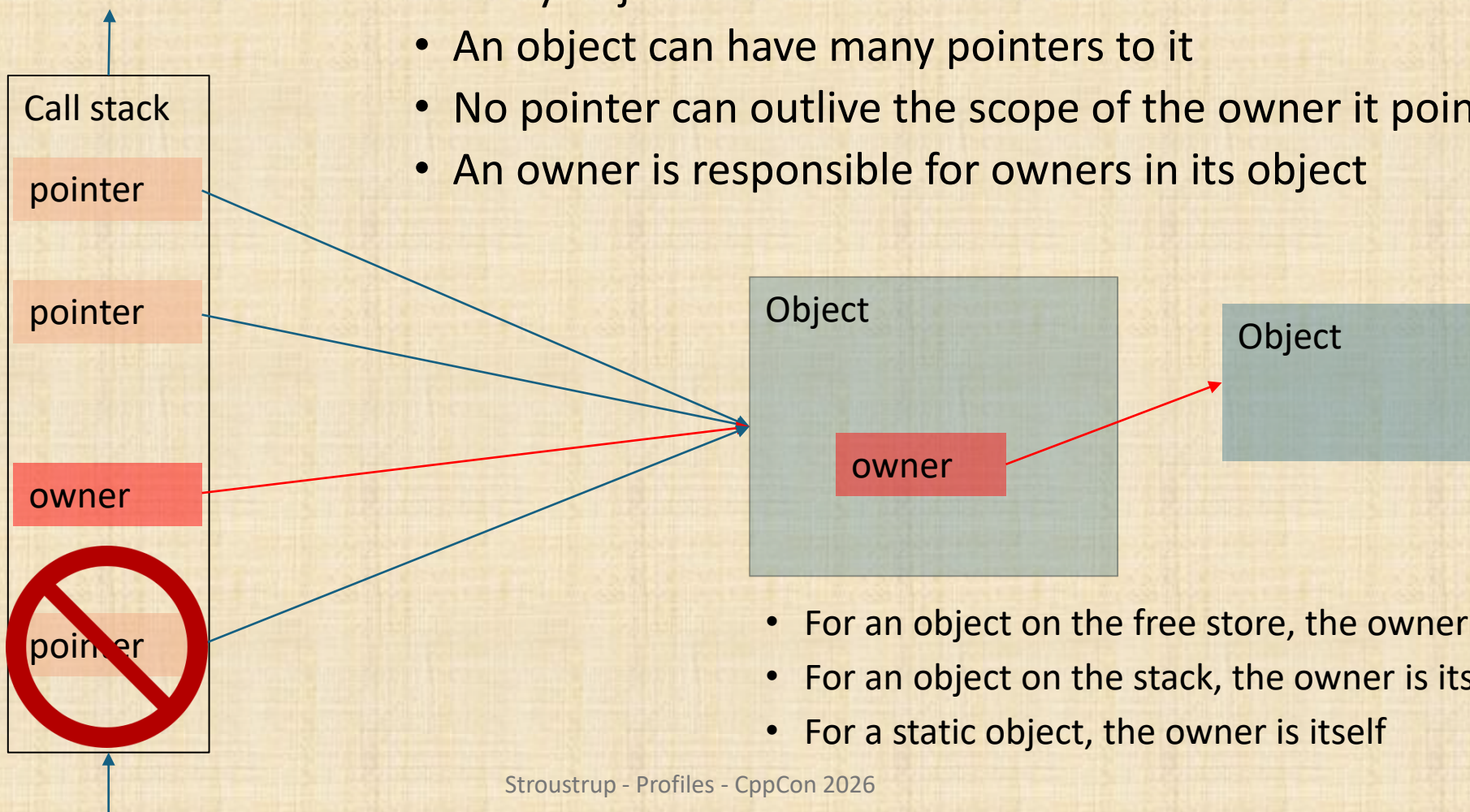
- In general, impossible
- Often very expensive

- Simplification

- Make static analysis conservative and cheap
- Consider every deletion unconditional
- Consider every access through a pointer unconditional
- If this is too conservative, the programmer can suppress the invalidation profile locally
- Now the 4 possible disasters and the leak are caught by the invalidation profile

Owners and “Pointers”

- Every object has one owner
- An object can have many pointers to it
- No pointer can outlive the scope of the owner it points to
- An owner is responsible for owners in its object



- For an object on the free store, the owner is a pointer
- For an object on the stack, the owner is itself
- For a static object, the owner is itself

What pointers can be used?

- Pointers to object guaranteed to be “live”

```
int glob = 5;
```

```
void f(int* p1)           // pointer argument
```

```
{
```

```
    int x = 7;
```

```
    int* p2 = &x;         // pointer to local
```

```
    int* p3 = new int{9};  // pointer to free store object (but other problem: potential leak)
```

```
    int* p4 = &glob;       // pointer to static
```

```
    // ... p1, p2, p3, and p4 are usable here ...
```

```
}
```

Invalidation through relocation

- Consider

```
void f(vector<int>& vi)
{
    vi.push_back(9);           // may relocate vi's elements
}

void g()
{
    vector<int> vi1 { 1,2 };
    f(vi1);                   // OK, no pointer to a vi1 element; reallocation is OK

    vector<int> vi2 { 1,2 };
    auto p = vi2.begin();     // point to first element of vi
    f(vi2);                   // likely disaster
    *p = 7;
}
```


Add... ▾ More ▾ Templates
Share ▾ Policies  ▾ Other ▾

C++ source #1 

A ▾

Save/Load

+ Add new... ▾

V Vim

 Cpplnsights

 Quick-bench

C++ ▾

```

1  [[profiles::enforce(std::invalidation)]];
2
3  #include<vector>
4  using std::vector;
5
6  void f(vector<int>& vi)
7  {
8      vi.push_back(9);    // may relocate vi's elements
9  }
10
11 void g()
12 {
13     vector<int> vi1{ 1,2 };
14     f(vi1);              // OK, no pointer to a vi1 element; real
15
16     vector<int> vi2{ 1,2 };
17     auto p = vi2.begin(); // point to first element of vi
18     f(vi2);
19     *p = 7;              // likely disaster
20 }
```

Output of x86-64 gcc (P4324 contracts) (Compiler #1)  X

A ▾

☒ Wrap lines

 Select all

```

<source>: In function 'void g()':
<source>:19:10: error: use of a value bound to 'vi2',
potentially invalidated by an earlier mutation of 'vi2',
not permitted under the 'std::invalidation' profile
19 |         *p = 7;          // likely disaster
    |         ^
Compiler returned: 1
```

x86-64 gcc (P4324 contracts) (Editor #1)  X

x86-64 gcc (P4324 contra ▾





Compiler options.. ▾

A ▾













```

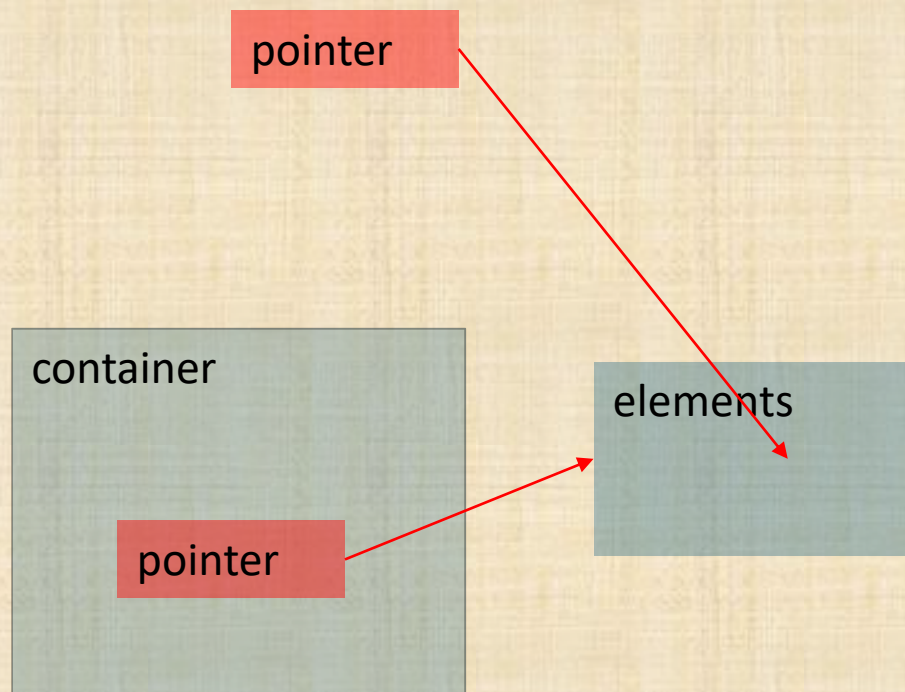
1  <Compilation failed>
2
```

What are “Containers”?

- Here, “container” means any object containing a pointer to elements that may be relocated
 - Classes with pointer members
 - Lambdas (they are classes, and remember capture-by-reference)
 - **jthreads** containing pointers (they are classes in a defined scope)
 - **unique_ptr**s and **shared_ptr**s (they are classes with a single logical pointer member)
 - Pointers to pointers
 - References to pointers
 - Arrays of pointer
- Note that a “container” can be a “pointer” and vice versa.

“Containers” and “Pointers”

- If the elements are relocated, a pointer to an element will dangle
 - That must be prevented



Invalidation through relocation

- Invalidation through relocation rules
 - Don't call a function that can invalidate if there exist a pointer to its element
 - **const** functions can't relocate aka invalidate
 - The invalidation profile assumes that non-**const** functions relocate
 - Similar for functions with **const** and non-**const** pointer arguments

Invalidation through relocation

- Relying just on const/non-const would lead to massive false positives
 - Many non-**const** functions do not invalidate
 - Proving that would be very costly
- To help the static analyzer (e.g., the compiler) use an attribute **[[not_invalidating]]**
 - Easily verified

```
class int_vec {  
    // ...  
    int& operator[] [[not_invalidating]] (int i);           // verified when defining  
    const int& operator[](int i) const;                   // doesn't invalidate  
  
    int size() const;   // doesn't invalidate  
};  
  
void f(vector<int> arg1 [[not_invalidating]], vector<int>& arg2);
```

What pointers may escape a scope?

- With the invalidation profile enforced, we can't pass a dangling pointer

```
int glob = 5;

void f(int* p1)           // pointer argument
{
    int x = 7;
    return &x;            // error: pointer to local; will dangle
    // ...
    return p1;            // OK: pointer passed in
    // ...
    return new int{9};     // error: pointer to free store object, but potential leak caught
    // ...
    return &glob;         // OK: pointer to static
    // ...
}
```

What pointers may escape a scope?

- Containers of pointers
 - considered “pointers” by the invalidation profile
 - E.g., vectors with pointer elements, smart pointers, lambdas with captures pointers, ...
 - Can contain any valid pointer
 - Can’t be returned if containing a pointer that would dangle

```
vector<int*> f(int* p)
{
    vector v1 { p };
    return v1;                // OK: p won't dangle

    int x = 7;
    vector v2 { p, &x };      // v2 usable here, but mustn't escape
    return v2;                // error: &x will dangle
}
```

What pointers may escape a scope?

- Real-world example

```
class WidgetFactory {                                // optionally log each creation
    Window* window_;
    Logger* logger_ = nullptr;
public:
    WidgetFactory(Window& w, Logger& l = nullptr) :window_(&w), logger_(&l) {}
    WidgetFactory(Window& w) :window_(&w){}

    Widget create_widget()
    {
        if (logger_) logger_>log("Creating new Widget");
        return Widget{*window_};                    // returns object containing a pointer
    }
};
```

- How do we know that the Widget returned doesn't contain a pointer that will dangle?
 - We don't, so that caller must manage

What pointers may escape a scope?

- Real-world example

```
Widget inner(Window& window)
{
    Logger log;
    WidgetFactory factory{window, log};           // factory holds pointers to window and log
    return factory.create_widget();               // error: the profile doesn't know that
                                                // the widget created doesn't depend on &log
}
```

- The analyzer can't know that the Widget returned doesn't contain a pointer to log
 - A WidgetFactory is a kind of container of pointers
 - It is OK to construct a container with pointers into different scopes
 - It is not OK to return a container with a pointer to a local
 - It is not OK to return a pointer from such a container, so by default that example is an error!

What pointers may escape a scope?

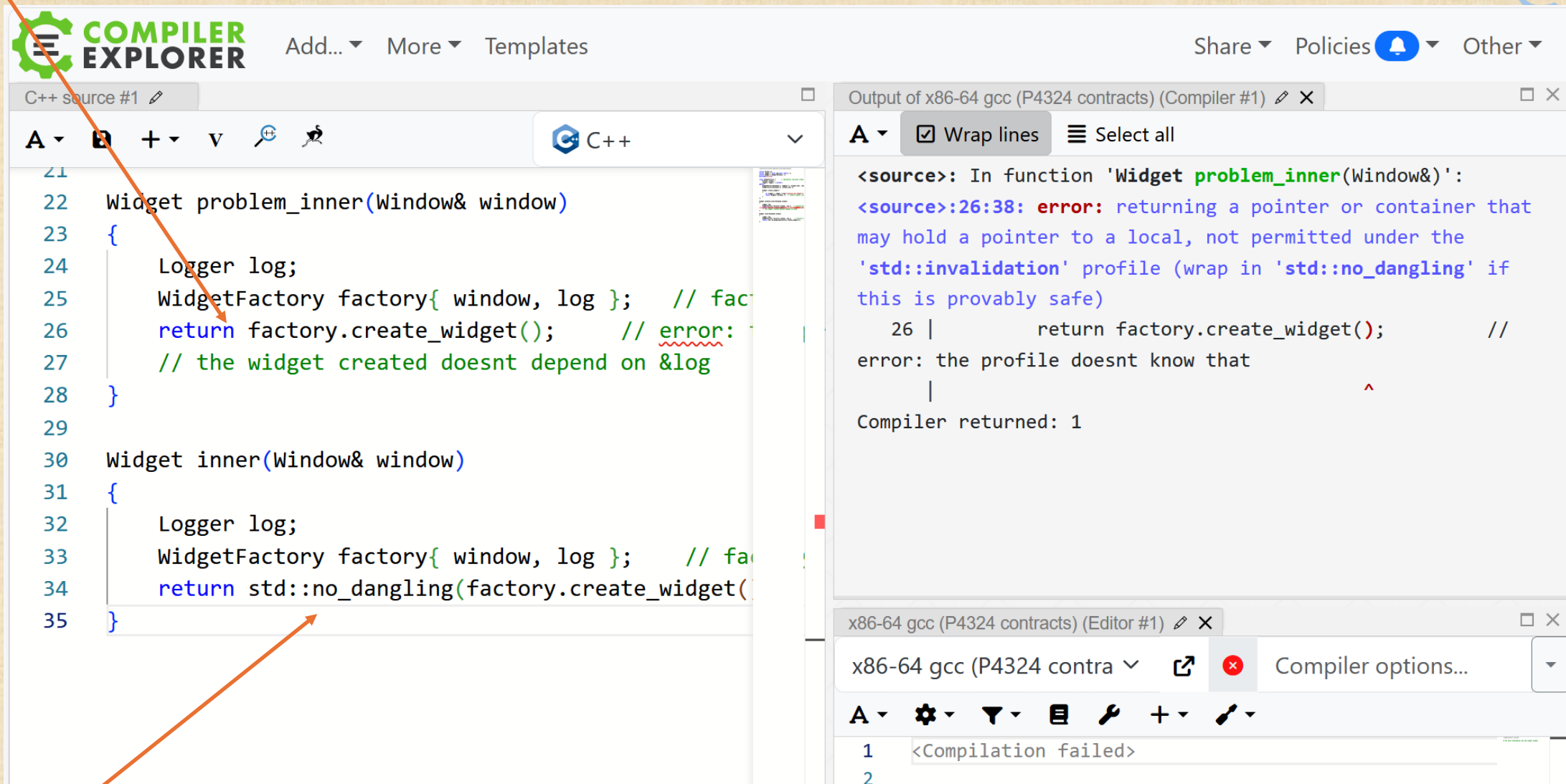
- In general, we can't statically determine whether a returned pointer from a container containing potentially dangling pointers is OK

```
Widget inner(Window& window)
{
    Logger log;
    WidgetFactory factory{window, log};           // factory holds pointers to window and log
    return no_dangling(factory.create_widget()); // the widget created doesn't depend on &log
}
```

Very specific profile suppression

- The **no_dangling()** asserts – without proof – that there will be no dangling
 - This is a local suppression of the invalidation profile
 - The alternative to **no_dangling()** is the suppression of the invalidation profile

Potential error



COMPILER EXPLORER Add... More Templates Share Policies Other

C++ source #1

```

21
22 Widget problem_inner(Window& window)
23 {
24     Logger log;
25     WidgetFactory factory{ window, log }; // fac
26     return factory.create_widget(); // error:
27     // the widget created doesnt depend on &log
28 }
29
30 Widget inner(Window& window)
31 {
32     Logger log;
33     WidgetFactory factory{ window, log }; // fa
34     return std::no_dangling(factory.create_widget(
35 }
  
```

Output of x86-64 gcc (P4324 contracts) (Compiler #1)

Wrap lines Select all

```

<source>: In function 'Widget problem_inner(Window&)':
<source>:26:38: error: returning a pointer or container that
may hold a pointer to a local, not permitted under the
'std::invalidation' profile (wrap in 'std::no_dangling' if
this is provably safe)
    26 |         return factory.create_widget(); //
error: the profile doesnt know that
      |
Compiler returned: 1
  
```

x86-64 gcc (P4324 contracts) (Editor #1)

x86-64 gcc (P4324 contra Compiler options...

```

1 <Compilation failed>
2
  
```

Suppress potential error

Preventing leaks

- Using **[[owner]]** to make the implementation of resource handles explicit
- Should preventing leaks be part of the invalidation profile?
 - Yes, because explicit **delete** in code can lead to dangling pointers

```
class Int_vec {  
    int* elem [[owner]];  
    int sz;  
public:  
    int_vec(int s) : elem{new int[s]}, sz{s} {}  
    ~int_vec() { delete[] elem; }  
}
```

[[owner]] rules

- An [[owner]] must be deleted or passed on to another [[owner]]
 - A non-[[owner]] mustn't be deleted
 - An [[owner]] assigned to a non-[[owner]] creates an alias
 - Assigning an [[owner]] to another [[owner]] transfers the obligation to delete and leaves behind a non-[[owner]]
-
- Note change of notation from earlier presentations
 - For consistency with other profiles

Representing ownership

- Consider

```
void use(int* p, int* q [[owner]])
{
    // ...
    delete p;           // error: not owner
    delete q;           // required
}

void f()
{
    int* p = new int{7}; // error: throws away [[owner]] attribute
    int* q [[owner]] = new int{7};
    use(p,q);
    *p = 7;
    delete q;           // error: [[owner]] responsibility passed to use()
}
```


Add... ▾ More ▾ Templates

C++ source #1

A ▾
+
+
v
🔍
🔗
C++

```

3 void use(int* p, int* q [[owner]] )
4 {
5     // ...
6     delete p;    // error: not owner
7     // delete q; // error: required
8 }
9
10 void f()
11 {
12     int* p = new int{ 7 };    // error: throws away [[owner]] a
13     int* q [[owner]] = new int{ 7 };
14     use(p, q);
15     *p = 7;
16     delete q;    // error: owner responsibility passed out
17 }

```

Output of x86-64 gcc (P4324 contracts) (Compiler #1)

A ▾
Wrap lines
Select all

```

<source>:3:23: error: '[[owner]]' pointer 'q' is never deleted or passed on
before the function returns, under the 'std::invalidation' profile
   3 | void use(int* p, int* q [[owner]] )
     |                               ~~~~~^~~~~~
<source>: In function 'void f()':
<source>:12:14: error: the freshly-allocated value assigned to 'p' is never
deleted or passed on before the function returns, under the
'std::invalidation' profile
   12 |         int* p = new int{ 7 };    // error: throws away
[[owner]] attribute
     |                               ^
<source>:16:9: error: 'q' is consumed again here, after already being
consumed on every path reaching this point, under the 'std::invalidation'
profile
   16 |         delete q;    // error: owner responsibility passed
out

```

x86-64 gcc (P4324 contracts) (Editor #1)

x86-64 gcc (P4324 contra
Compiler options...

A ▾
⚙️
🔍
📄
🔗
+
🔗

1
<Compilation failed>

How do we represent ownership?

- High-level
 - Use an ownership abstraction

Stay high-level when you can

- Low-level
 - mark owning pointers **[[owner]]**

For implementing ownership abstractions
And for C-style interfaces

- Note
 - I talk about “pointers”
 - What I say applies to anything that refers to an object
 - References
 - Containers of pointers
 - Smart pointers
 - Pointers to pointers
 - Lambda captures
 - ..

Invalidation profile summary

- An invalidating function for a container can't be called if pointer to an element has been taken
- A non-**const** function is assumed to invalidate
 - A **const** function doesn't invalidate
- A function is assumed to invalidate a non-**const** argument
 - A function doesn't invalidate a **const** argument
- Invalidation profile attributes
 - **[[not_invalidating]]** this argument will not be invalidated by this function
 - **[[owner]]** this pointer must be deleted or passed on
- An attribute always appears after the name of the name it refers to
- A function to assert that its argument doesn't contain a dangling pointer
 - **no_dangling()** *an intrinsic function*
- Zero run-time overhead



The ranges profile

- Guarantee
 - All range errors caught at compile-time or run-time
 - All nullptr errors caught at compile-time or run-time
 - All run-time violations are handled as specified by the profile
- We already have efficient hardened standard libraries to check proper uses
- A profile's run-time checks can't be suppressed except through profile suppression
- We need static analysis to eliminate non-checkable uses

```
void f(int i, span<int> s, int*p)
{
    int x = s[i];           // OK: checked at run time (or in constexpr cases maybe at compile time)
    int y = p[i];           // error: can't be checked
}
```



The grand scheme

- The end goals
 - There are more than one goal
 - Some goals are domain specific
 - “Memory safety” by itself is not good enough
- Where to start
 - The framework
 - The initialization, invalidation, and ranges profiles
 - Catch the most common bugs at minimal cost
 - Have a plan for further mutually-supporting profiles
- Constraints
 - No loss of expressiveness
 - No garbage collector
 - Hardly any profile suppression
 - Zero overhead
 - No change of base language

Coherent guarantees

Not just collections
of added checks in code

My favorite: Type-and-resource safety

- Every object is accessed according to the type with which it was defined (type safety)
- Every object is properly constructed and destroyed (resource safety)
- Every access through a pointer is to a valid object or the **nullptr** (memory safety)
- Every access through a pointer is not through the **nullptr** (often a run-time check)
- Every access through a subscripted pointer is in-range (often a run-time check)

- That is

- just what C++ requires (also C)
 - what most programmers have tried to ensure since the dawn of time
- The enforcement rules are more deduced than invented

Enforcement rules are
mutually dependent and
mutually supportive.
Don't judge individual rules in isolation

Notions of what constitutes safety violations

- **Logic errors:** perfectly legal constructs that don't reflect the programmer's intent, such as using `<` where a `<=` or a `>` was intended.
- **Resource leaks:** failing to delete resources (e.g., memory, file handles, and locks) potentially leading to the program grinding to a halt because of lack of available resources.
- **Concurrency errors:** failing to correctly take current activities into account leading to (typically) obscure problems (such as data races and deadlocks).
- **Memory corruption:** for example, through the result of a range error or by accessing and memory through a pointer to an object that no longer exists thereby changing a different object.
- **Type errors:** for example, using the result of an inappropriate cast or accessing a union through a member different from the one through which it was written.
- **Overflows and unanticipated conversions:** For example, an unanticipated wraparound of an unsigned integer loop variable or a narrowing conversion.
- **Timing errors:** for example, delivering a result in 1.2ms to a device supposedly responding to an external event in 1ms.
- **Termination errors:** a library that terminates in case of “unanticipated conditions” being part of a program that is not allowed to unconditionally terminate.

Profiles

- Can save us from the complexities caused by
 - Decades of evolution
 - Massive growth in application areas
- Can close the major safety holes
 - Without breaking the most effective design and implementation techniques
 - While often adding efficiency
- Remaining safety holes will exist primarily for reasons of compatibility
 - Will be explicit in code
 - To allow gradual adoption
 - Explicit so as to simplify code review
 - Specific, not all or nothing
 - Hardware and operating system access

The initial set of profiles considered for the standard

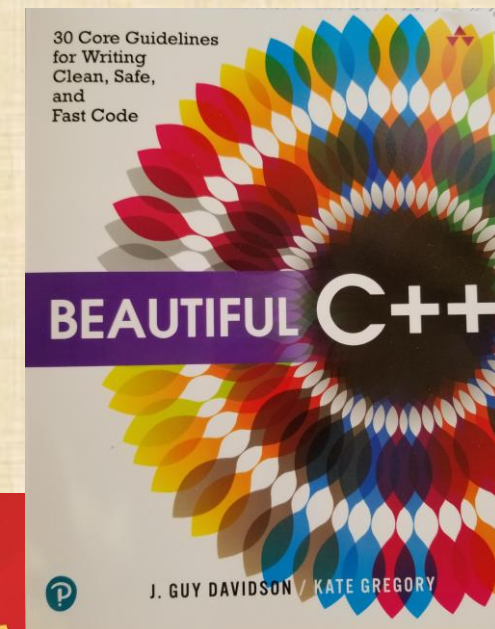
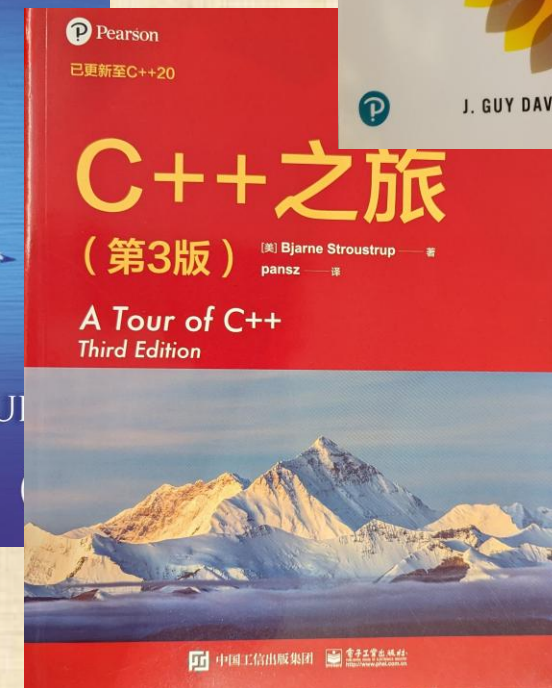
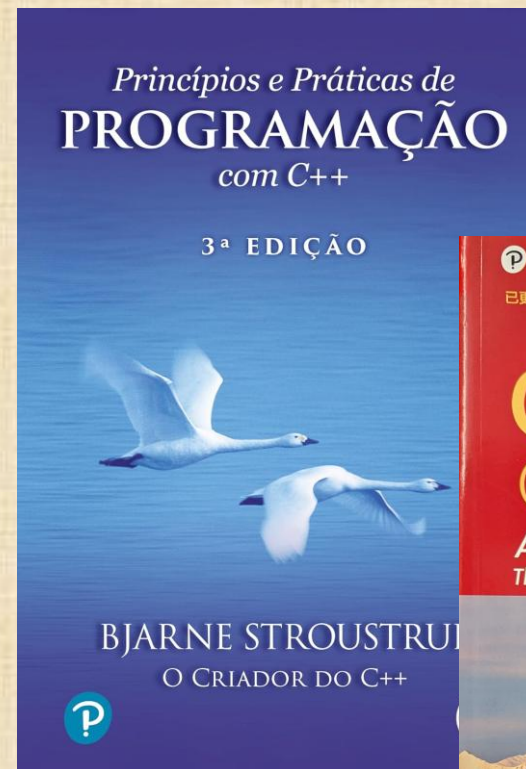
- **Algorithms** – all ranges, no dereferences of end()
- **Arithmetic** – catch overflow and underflow, catch implicit narrowing conversions
- **Casting** – ban them
- **Concurrency** – eliminate deadlocks and data races (hard)
- **Initialization** – every object initialized
- **Invalidation** – no access through invalidated pointer (incl. dangling pointers)
- **Pointers** – no subscripting of built-in pointers (use span, vector, string, etc.)
- **Ranges** – range errors caught
- **RAII** – every resource owned by a handle
- **Type** – initialization, ranges, casting, invalidation, pointers
- **Union** – no unions (use variant, etc.)
- A profile must be explicitly requested and can be suppressed where not appropriate

Design constraints

- Keep it simple
 - If necessary, suppress profile rather than complicating guarantees
- Enable gradual introduction
- We need solid experience
- Rely on principled approaches
- Distinguish guarantees from implementation of their enforcement
 - The guarantees are simple
 - Their enforcement is usually complex
- We need a variety of run-time violation responses
 - Specified by the profile
 - Guaranteed to be enforced
- Compatibility: no change of semantics
 - Except by specifying what happens to (former) UB
- Zero overhead

Long-term aims

- Use profiles (and other language and library improvements) to
 - Raise the level of abstraction
 - Allow the compiler to catch more errors
- To eliminate whole classes of errors
- To simplify system building
- To simplify tool building
- To improve performance
- To simplify education and training



Profiles implementations



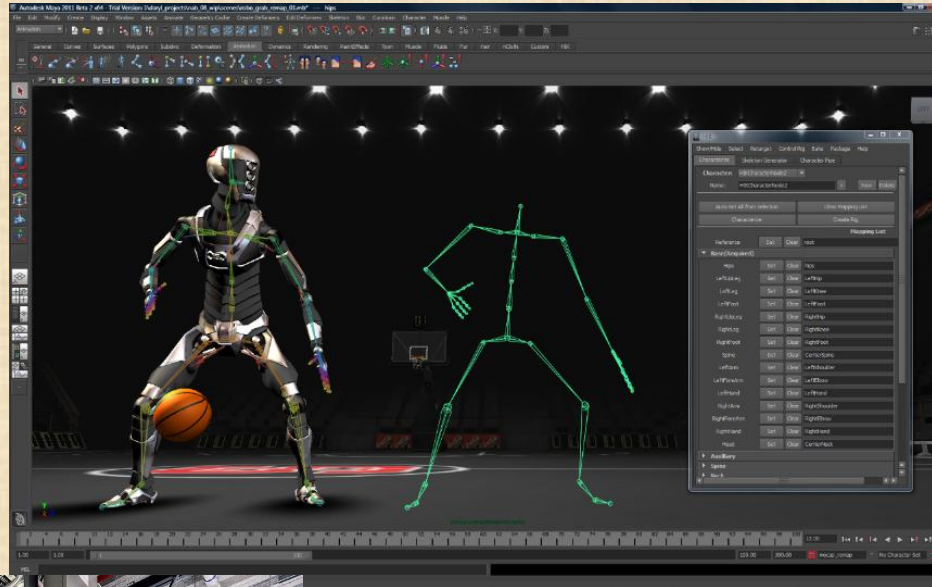
Ville Voutilainen

- Compilers
 - GCC: <https://godbolt.org/z/hb7e47v3d>
repo <https://github.com/villevoutilainen/gcc/tree/p4324>
 - Clang: <https://godbolt.org/z/xMox44aM8>
repo <https://github.com/cppalliance/clang/tree/profiles-init>
 - And a third effort: <https://6it.dev/>
- Documentation
 - What are profiles. P3704R0
 - Design Alternatives for Type-and-Resource Safe C++. P2687R0
 - Safety Profiles: Type-and-resource Safe programming in ISO Standard C++. P2816R0
 - Concrete suggestions for initial Profiles. P3038R0
 - A framework for Profiles development. P3274R2
 - Clang profiles framework
 - Profile invalidation – eliminating dangling pointers. P3446R0. 2024-10-14. R2 and implementations will match.
 - An initialization profile. P4222R2. 2026-08-12. R3 and implementations will match.



Krystian Stasiowski

Profiles will change the way C++ is used, taught, and perceived



AMADEUS

Stroustrup - Profiles - CppCon 2026

More information about Profiles

- The basic model
 - B. Stroustrup, H. Sutter, and G. Dos Reis: [A brief introduction to C++'s model for type- and resource-safety](#). Isocpp.org. October 2015. Revised December 2015.
 - B. Stroustrup: [Type-and-resource safety in modern C++](#). SG22/WG21. P2410r0. 2021-07-12.
 - And many papers on WG21
- Videos
 - YouTube: e.g., Bjarne Stroustrup, Herb Sutter, Gabriel Dos Reis
- Books
 - B. Stroustrup: [A Tour of C++ \(third edition\)](#)
 - G. Davidson and K. Gregory: [Beautiful C++: 30 Core Guidelines for Writing Clean, Safe, and Fast Code](#)
 - Rainer Grimm: [C++ Core Guidelines Explained: Best Practices for Modern C++](#)